```
  object(Closure)#2 (0) {
  }
}
Fatal error: Uncaught Error: Call to undefined method
stdClass::getMusicTaste()
```

Adding the anonymous function is fine (notice that the object property `getMusicTaste` is an instance of the `Closure` class, which is how PHP stores anonymous function internally), and it becomes another property of the object. Calling a property in this way is not allowed, despite the fact that it is a `callable` type. Them's the rules I'm afraid. There are tricks that can be pulled to execute the closure we added to `$lebowski`, including one we've seen already in Chapter 3, "Functions": the self-invoking function:

```
($lebowski->getMusicTaste)();

Output:
Fatal error: Uncaught Error: Using $this when not in object context...
```

Now we've managed to call the "new method," but the result is still a fatal error. This is because `getMusicTaste()` is still not an object method; it is simply a `Closure` object that `$lewbowski->getMusicTaste()` is a reference to. The clue is in the error message: "not in object context." The `$this` magic variable only exists under some very exact circumstances; details are coming up.

Cloned Objects

Another way to make a new object is by cloning an existing one. Simply use the `clone` keyword:

```
$lebowski = new stdClass();
$lebowski->hates = 'The Eagles';

$theOtherLebowski = clone $lebowski;
var_dump($theOtherLebowski);
```